

Book-fetch Database Final Report

DASC 450 – Database Design

Phase 5: Final Consolidated Submission

Team: Collin Ryan, Kevin Mendez Rivera, Jair Vargas, Nate Agbemadon, Steven Schmitz

Date: *12/10/2025*

1. Application Overview & Architecture Summary

1.1 Purpose

Book Fetch is a university book management system that allows students, instructors, admins, and customer-support staff to manage academic books, carts, orders, reviews, course materials, and help-desk tickets. The system improves book purchasing and academic resource management.

1.2 Core User Roles

- Students – create accounts, manage carts, purchase books, review titles.
- Administrators – manage inventory, resolve tickets, oversee employee actions.
- Customer Support Employees – create/triage tickets, assist students.
- Super Admins – audit employee actions, manage roles, access all reports.

1.3 System Architecture

Frontend: Python + Tkinter

The UI was built using Tkinter, giving us windows for:

- Student creation
- Cart creation and management
- Adding items to cart
- Placing orders
- Updating cart quantities
- Cancelling orders
- Adding book reviews
- Running database reports

Application logic:

MainApp handles navigation and DB health checks.

StudentModule manages student CRUD, carts, orders, and reviews.

ReportsModule dynamically renders SQL report outputs in a TreeView grid.

db.py manages MySQL connection pooling

Backend: MySQL

The database gives us tables that support the following:

- Student information
- Employees, roles, audit logs
- Books, authors, categories, subcategories, keywords
- Reviews
- Carts, orders, and purchase history
- Courses and academic book assignments
- Ticketing and ticket history

2. Libraries / Drivers Used

Python Libraries

- tkinter — GUI framework
- tkinter.ttk — TreeView tables for displaying report results
- messagebox — input validation & error prompts
- re — email/date validation
- platform — OS-specific scrolling behavior
- mysql.connector — database driver
- mysql.connector.Error — error handling for SQL operations

MySQL Components

- MySQL Server 8.x
- SQL scripts: create_tables.sql, drop_tables.sql, insert_demo_data.sql, delete_demo_data.sql

Driver

- mysql-connector-python

3. Description of Each UI Module (with Screenshots)

4. Queries / Reports Mapping + Sample Outputs

4.1 Queries Used by Student Module

Create Student

```
INSERT INTO student (username, university_id, Student_year, student_status,  
                    phone, major, last_name, first_name, email, birth_date, address_id)  
VALUES (%s, ...)
```

Purpose: Add a new student profile.

Sample Output: “Student created successfully!”

Create Cart

```
INSERT INTO cart (student_id, cart_status)
```

```
VALUES (%s, 'Active')
```

Purpose: Initialize cart for a student.

Add Book to Cart

```
INSERT INTO cart_item (cart_id, book_id, quantity, unit_price)
```

```
VALUES (%s, %s, %s, %s)
```

```
UPDATE book SET quantity = quantity - %s WHERE book_id = %s
```

Purpose: Add items while reducing inventory.

Place Order

```
INSERT INTO customer_order (student_id, shipping_type, credit_card_number,
```

```
    credit_card_exp, credit_card_name, credit_card_type, order_status)
```

```
VALUES (%s, ..., 'Paid')
```

```
INSERT INTO order_item (order_id, book_id, quantity, unit_price)
```

```
INSERT INTO purchase_history (student_id, book_id, order_id)
```

Sample Output Table (Order Item Details):

Book ID	Quantity	Unit Price
---------	----------	------------

1	1	120.00
---	---	--------

4	1	110.00
---	---	--------

Update Cart Item

```
UPDATE cart_item SET quantity = %s WHERE cart_id = %s AND book_id = %s
```

```
UPDATE book SET quantity = quantity -/+ %s WHERE book_id = %s
```

Cancel Order

```
UPDATE book SET quantity = quantity + %s WHERE book_id = %s
```

```
UPDATE customer_order SET order_status = 'Cancelled'
```

Review Submission

```
INSERT INTO review (student_id, book_id, rating)
```

```
VALUES (%s, %s, %s)
```

4.2 Queries Used in the Reports Module

Reports module supports:

Report 1 – Students at St Thomas University

From *reports_module.py*

Full SQL in *reports.sql*

Sample Output:

student_id	username	first_name	last_name	major	university_name
1	jpork	John	Pork	Computer Science	St Thomas University

Report 2 – Graduate Students

SQL in both files.

Sample Output:

student_id	first_name	last_name	student_status
17	Sarah	Kim	Graduate

6. Notes on Deviations / Changes From Earlier Designs

Constraint changes

- CHECK constraints on:
 - quantity ≥ 0
 - valid enums for condition, format, acquisition_type
 - rating between 1 and 5
- UNIQUE constraints:
 - username
 - email
 - isbn13/isbn10
 - several junction tables

Naming Conventions

- Converted all tables and columns to snake_case.
- Renamed:
 - Order → customer_order
 - OrderItem → order_item

Foreign Key Changes

- Added ON DELETE CASCADE / RESTRICT conditions to enforce integrity.
- Simplified purchase history with new surrogate PK.

Appendix:

Phase 1:

1. In 3–5 sentences: What problem are you solving and for whom?

Steven:

This project helps Employees keep track of the Students that have can rent and buy books. It helps them manage their bookstore inventory by see how many of each book they have. They can see and track reviews on the books they have, issues (tickets), and book cancellations. They can manage access to the book store and employee access.

2. Who Will Use It & What They Need (User Stories) Write simple “who/what/why” sentences; no jargon.

Steven:

Users: Students, Employees (Customer Support, Administrators)

Customer Support (CS): log technical issues with the website, perform order cancellations, and log general complaints.

Administrators (Admin): handle trouble tickets assigned by CS or submitted by students, adjust inventory, and add university course book requirements.

Super Administrator: can create/deactivate CS and Admin accounts

Students: Can rent and buy books, submit problem tickets, submit reviews on books

3. What the System Should Do (Functional Requirements) List concrete actions the system should support — start sentences with “The system should ...”

Steven:

The system should allow students to order books (add books to cart), submit tickets, and reviews.

The system should allow administrators to add new books to inventory, add new universities, departments, courses, and book associations.

The system should allow Super administrators to add new employees and deactivate users.

The system should be able to generate various reports on students, employees, and universities.

4. Qualities & Limits (Non-Functional Requirements) These describe how well your system should perform. Pick 4–8 that matter.

(Nate)

Performance: Search results should appear within 3 seconds of a query

Availability: The system should always be available (unless in maintenance shutdown)

Scalability: Should have the capacity to support 10k books and 5k active students.

Security: Only authorized users should be permitted to modify the inventory or the tickets. Students have limited access, meaning they only have permission limited to their own student account.

Usability: Should be intuitive. Users should be able to effectively accomplish their task efficiently.

Compatibility: Should be able to run on any common browser

Integrity: Tickets/orders/inventory should be validated to prevent duplicates.

Examples:

Search results should appear within two seconds.

The system should be available 24/7

Only authorized users can edit or delete records.

System should run on MySQL and support up to 10,000 records.

5. Assumptions & Not Doing (For Now) Collin Ryan

Assumptions: All data given is in its correct row/column. No additional data collection is needed. All data shall be publicly available.

Out of Scope: Regular maintenance. Front end design. Setting up or maintaining servers. Encryption or any other kind of data security.

6. Rough Plan (Next Steps) Example outline: “Design an E-R diagram → create relational tables in MySQL → populate with sample car and sales data → develop a simple Python/Java interface → test queries for adding and retrieving data.”

Include the tools you expect to use (MySQL, Python, or Java).

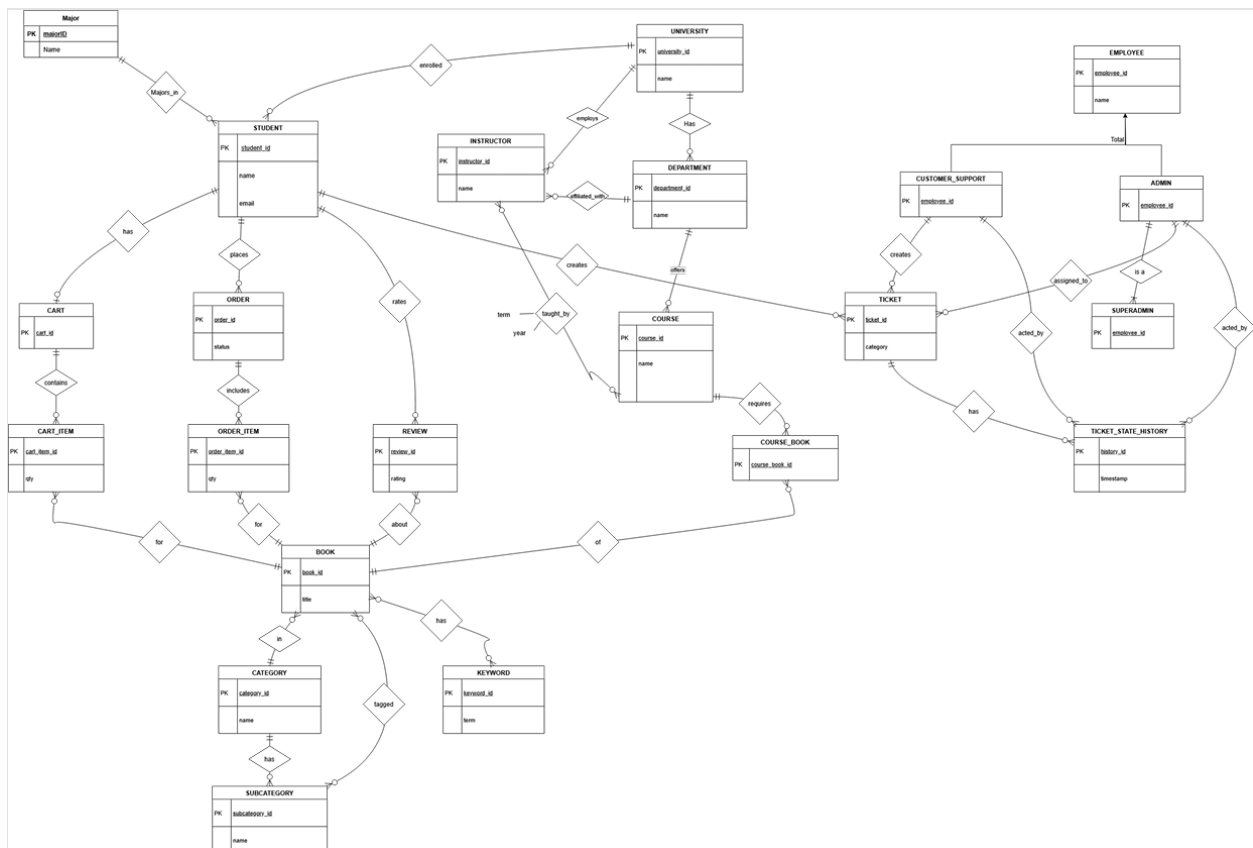
Design an E-R diagram → create relational tables in MySQL → populate with sample car and sales data → develop a simple Python/Java interface → test queries for adding and retrieving data.

7. Questions for the Instructor Ask anything unclear now to avoid redesign later.

“Do we need to include used car trade-ins in Phase 1?”

“Should reports be generated in CSV format or displayed on screen?”

Phase 2:



Relationships:

- Each university has many departments; each department belongs to one university.
(University-Department, 1:N)
- Each university can have many students, each student belongs to one university.
(University-Student, 1:M)
- Each department offers multiple courses; each course belongs to one department.
(Department-Course, 1:N)
- Each university has many instructors; each instructor is affiliated with one university.
(University-Instructor, 1:N)
- Each department has many instructors; each instructor belongs to one department.
(Department-Instructor, 1:N)
- Each course can have multiple instructors; each instructor can teach multiple courses.
(Course-Instructor, M:N)
- Each course can have multiple books; each book can be associated with multiple courses.
(Course-Book, M:N)
- Each book belongs to one category.
(Book-Category, N:1)
- Each category can have many subcategories; each subcategory belongs to one category. (Category-Subcategory, 1:N)
- Each book can belong to zero or more subcategories; each subcategory can include many books. (Book-Subcategory, M:N)
- Each book can have zero or more keywords; each keyword can be linked to many books. (Book-Keyword, M:N)
- Each student can create multiple reviews; each review belongs to one book
(Student-Review, 1:N)
- Each student can have at most one active cart.
(Student-Cart, 1:0 .. 1)
- Each cart can contain multiple cart items.
(Cart-CartItem, 1:N)
- Each cart item refers to one book.
(CartItem-Book, N:1)
- Each cart becomes one order when a purchase is made.

(Cart-Order, 1:1)

- Each student can place multiple orders.
(Student-Order, 1:N)
- Each order can include multiple order items.
(Order-OrderItem, 1:N)
- Each order item refers to one book.
(OrderItem-Book, N:1)
- Each order is tied to one student for recommendation and purchase history.
(Order-Student, N:1)
- Each employee can be either a Customer Support agent or an Admin.
(Employee-CustomerSupport/Admin, ISA)
- Each Super Admin is a specialized Admin with higher privileges.
(Admin-SuperAdmin, ISA)
- Each trouble ticket belongs to one site category (user profile, products, cart, orders, or others).
(Ticket-TicketCategory, 1:1)
- Each ticket is created by either a student or a Customer Support employee (but not both).
(Ticket-Student/CustomerSupport, XOR 1:1)
- Each ticket can be assigned to one Admin.
(Ticket-Admin, 1:0 .. 1)
- Each ticket can go through multiple state changes.
(Ticket-TicketStateHistory, 1:N)
- Each ticket state change is performed by either an Admin or a Customer Support employee.
(TicketStateHistory-Admin/CustomerSupport, N:1)
- Each student is affiliated with one university and one major.
(Student-University/Major, N:1)

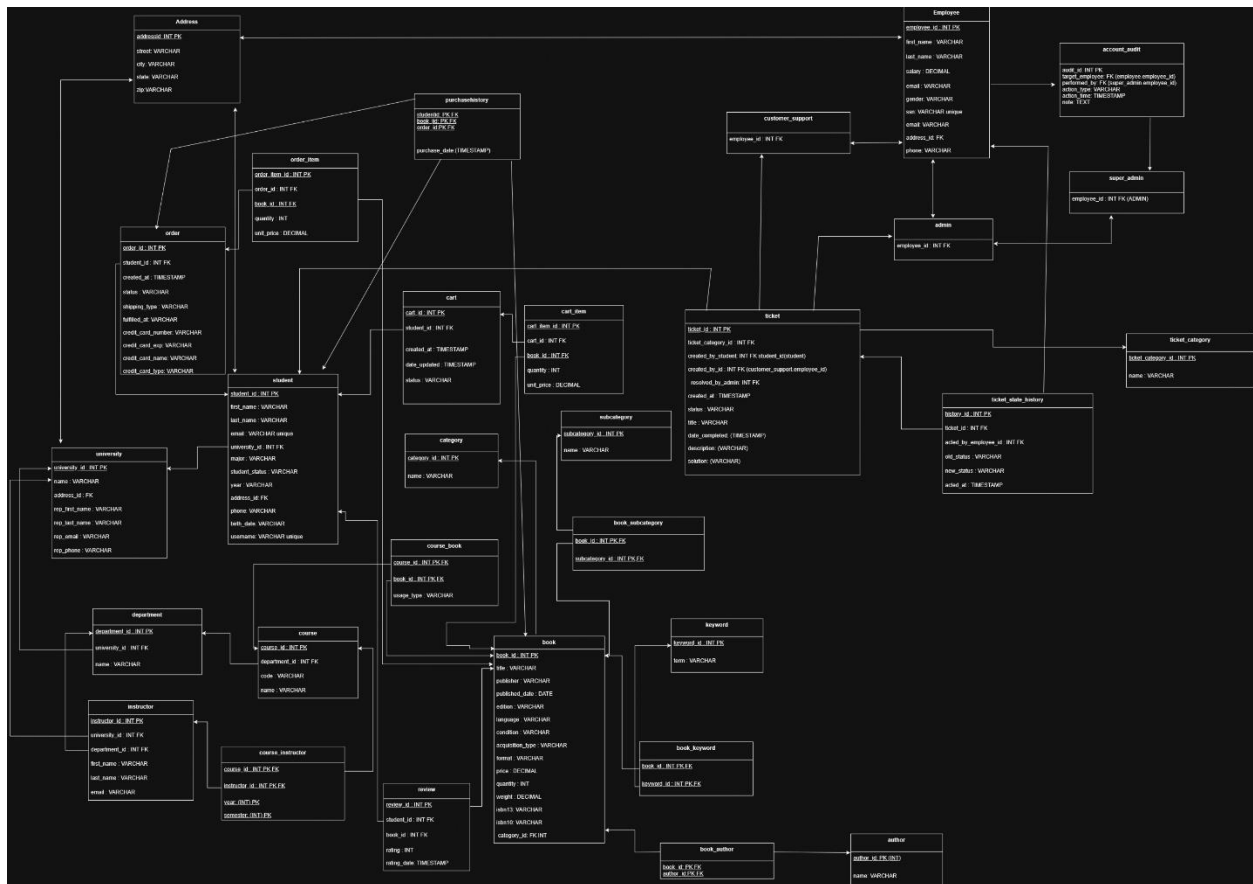
Assumptions:

- Each student is affiliated with one university and one major.
- Each student can have only one active cart at a time; past carts become inactive once an order is placed.
- Orders cannot be returned, but cancellations are supported if the order status allows.
- Each cart converts to exactly one order when a purchase or rental is completed.
- Each book belongs to exactly one category, but may belong to multiple

subcategories for finer classification.

- Each subcategory belongs to exactly one category (hierarchical organization).
 - Keywords are free-form tags associated with books for search and recommendation; they are not hierarchical.
 - Each course belongs to one department and one university, and can be linked to multiple books and multiple instructors.
 - The Course-Book relationship is time-specific (semester and year) but those details are captured in attributes, not as separate entities.
 - Employees (Customer Support, Admin, and Super Admin) share a common Employee superclass to avoid redundancy.
 - A Super Admin is a specialized Admin with higher privileges, not an independent entity.
 - Tickets are created by either a student or a Customer Support employee - never both.
 - Tickets can be assigned to one Admin for resolution.
 - Ticket state history records all changes in ticket status, along with the employee responsible for each change.
 - Book reviews are numeric ratings only (1-5 scale), with no text required unless extended in future versions.
 - The system does not model returns, refunds, or shipping logistics beyond the order's shipping type and status.
 - Recommendations are generated based on past purchases, book categories, keywords, and ratings above 3 - this is handled algorithmically outside the ER model.
 - Each university may have multiple departments, courses, and instructors; however, all are logically grouped under that university for organizational consistency.
 - The ER model represents conceptual relationships only - detailed attributes (e.g., payment info, timestamps) are deferred to the logical design phase.
 - The many to many relationship (Instructor->Course) was handled by adding attributes to the relationship entity (taught by) to remove redundancy if a course is offered in different semesters/years). Drawback includes having to join multiple tables to find out the term/year of a course.
-

Diagram:



Design Report:

Include:

2. Overview: Brief restatement of scope and key entities/relationships.

Schema Specification: Table-by-table description (attributes, PK, FK, notable constraints; brief data-type intent).

Student Table

Primary Key: **student_id (INT)**

Foreign Keys: **university_id (INT), address_id**

Attributes:

first_name (VARCHAR), last_name (VARCHAR), email (VARCHAR, unique), phone (VARCHAR, unique), birth_date (VARCHAR), major (VARCHAR), student_status (VARCHAR), year (VARCHAR), username (VARCHAR, unique)

Constraints:

Customer_Support

Primary Key: **employee_id (INT)**

Attributes:

Employee

Primary Key: **employee_id PK (INT)**

Foreign Key: **address_id**

Attributes:

first_name (VARCHAR), last_name (VARCHAR), gender (VARCHAR), salary (DECIMAL), ssn (VARCHAR, unique), email (VARCHAR, unique), phone (VARCHAR, unique)

Notable Constraints:

Same structure as Customer_Support, Admins can manage inventory, courses, update tickets

Super_Admin

Primary Key: **employee_id FK (ADMIN)**

Attributes:

Notable Constraints:

Exactly one Super Admin, Super Admin can create/deactivate employees

Account_audit

Primary Key: **audit_id (INT)**

Foreign Keys: **target_employee (employee.employee_id), performed_by (super_admin.employee_id)**

Attributes:

action_type (VARCHAR), action_time (TIMESTAMP), note (TEXT)

Book

Primary Key: **book_id (INT)**

Foreign Key: **category_id**

Attributes:

title (VARCHAR), publisher (VARCHAR), published_date (VARCHAR), edition (INT), language (VARCHAR), isbn13 (CHAR13, unique), isbn10 (CHAR10, optional, unique), condition (VARCHAR), acquisition_type (VARCHAR), price (DECIMAL), quantity (INT), weight (DECIMAL), format (VARCHAR)

Notable Constraints:

isbn13 unique, quantity ≥ 0 , category required

Category

Primary Key: **category_id (INT)**

Attributes: name (VARCHAR)

Author

Primary Key: **author_id PK (INT)**

Attributes: name (VARCHAR)

BookAuthor

Primary Key: **(book_id, author_id)**

Foreign Keys: **book_id, author_id** FK

Other Attributes: none

Notable Constraints:

Composite PK(book_id, author_id). Many-to-many relationship

Subcategory

Primary Key: **subcategory_id** PK (INT)

Attributes: name (VARCHAR)

BookSubcategory

Primary Key: **(book_id, subcategory_id)**

Foreign Keys: **book_id, subcategory_id**

Notable Constraints:

Composite PK(book_id, subcategory_id). Many-to-many relationship

Keyword

Primary Key: **keyword_id** (INT)

Attributes: term (VARCHAR)

BookKeyword

Primary Key: **(book_id, keyword_id)**

Foreign Keys: **book_id, keyword_id**

Notable Constraints:

Composite PK(book_id, keyword_id). Used for recommendation

Review

Primary Key: **review_id** PK (INT)

Foreign Keys: **student_id** FK, **book_id** FK

Attributes:

rating (INT), rating_date (TIMESTAMP)

Notable Constraints:

rating must be between 1 and 5

Course_Book

Primary Key: **(course_id, book_id)**

Foreign Keys: **course_id, book_id**

Attributes: usage_type (VARCHAR)

Notable Constraints:

Composite PK(course_id, book_id). Supports listing required course materials. Needs several joins to get information on professor teaching certain courses with certain required books.

University

Primary Key: **university_id PK (INT)**

Foreign Key: **address_id**

Attributes:

name (VARCHAR), rep_first_name (VARCHAR), rep_last_name (VARCHAR), rep_email (VARCHAR), rep_phone (VARCHAR)

Department

Primary Key: **department_id (INT)**

Foreign Keys: **university_id**

Attributes: name (VARCHAR)

Instructor

Primary Key: **instructor_id (INT)**

Foreign Keys: **university_id, department_id**

Attributes:

first_name (VARCHAR), last_name (VARCHAR), email (VARCHAR)

Course

Primary Key: **course_id (INT)**

Foreign Keys: **department_id**

Attributes: name (VARCHAR), code (VARCHAR)

Course_Instructor

Primary Key: **(course_id, instructor_id, year, semester)**

Foreign Keys: **course_id, instructor_id**

Attributes:

year (INT), semester (VARCHAR)

Notable Constraints:

Composite PK (course_id, instructor_id, year, semester)

Ticket

Primary Key: **ticket_id PK (INT)**

Foreign Keys:

category_id, created_by_student: student_id (student),

created_by_cs (employee_id), resolved_by_admin (admin employee_id)

Attributes:

created_at (DATE/TIMESTAMP), title (VARCHAR), date_completed (DATE, nullable),

description (VARCHAR), solution (VARCHAR), status (VARCHAR)

Ticket_category

Primary Key: **ticket_category_id (INT)**

Attributes: name (VARCHAR)

Ticket_state_history

Primary Key: **history_id (INT)**

Foreign Keys: **ticket_id, employee_id**

Attributes: old_status (VARCHAR), new_status (VARCHAR), acted_at (TIMESTAMP)

Cart

Primary Key: **cart_id (INT)**

Foreign Keys: **student_id FK**

Attributes:

date_created (VARCHAR), date_updated (VARCHAR), status (VARCHAR)

Notable Constraints:

Student may have only one active/open cart

Cart_item

Primary Key: **(cart_id, book_id)**

Foreign Keys: **cart_id, book_id**

Attributes:

quantity (INT), unit_price

Notable Constraints:

Composite PK(cart_id, book_id). Quantity ≥ 0

Order

Primary Key: **order_id (INT)**

Foreign Keys: **student_id FK**

Attributes:

created_at (TIMESTAMP), fulfilled_at (VARCHAR, nullable), shipping_type (VARCHAR), credit_card_number (VARCHAR), credit_card_exp (VARCHAR), credit_card_name (VARCHAR), credit_card_type (VARCHAR), order_status (VARCHAR)

Notable Constraints:

Order originates from a cart checkout. No returns allowed; only cancellations

OrderItem

Primary Key: **(order_id, book_id)**

Foreign Keys: **order_id, book_id**

Attributes:

quantity (INT), unit_price (Decimal)

Notable Constraints:

Composite PK(order_id, book_id)

PurchaseHistory

Primary Key: **(student_id, book_id, order_id, purchase_date)**

Foreign Keys: **student_id, book_id**

Attributes:

purchase_date (TIMESTAMP)

Address

Primary Key: **address_id (INT)**

Attributes:

street (VARCHAR), city (VARCHAR), state (VARCHAR), zip (VARCHAR)

Normalization Rationale: How you achieved 3NF/BCNF (or why certain choices were kept); show at least one concrete normalization example (e.g., resolving partial dependency).

The database is designed in 3NF/BCNF. Each table has a primary key, and all attributes depend on that key. There are no partial or transitive dependencies.

For example, the Student table references Address using address_id rather than storing street, city, state, and zip directly. This avoids repeating address data for multiple students.

Many-to-many relationships like books and authors are handled with junction tables such as BookAuthor to prevent duplication and maintain normalization.

Constraints & Business Rules: Cardinalities enforced; integrity constraints (referential actions, uniqueness).

Any many to many relationship was turned into its own table other the best fit table used the primary key/foreign key.

Integrity rules:

- Foreign keys enforce referential integrity.
 - Unique fields include email, SSN, and ISBN13.
 - Quantities and ratings have limits (e.g., quantity ≥ 0 , rating 1-5).
-
- Trace to Requirements: Short mapping that shows how core user requirements are supported.
 - User requirements are shown in the table via the arrows and the data in each table.
 - Student module: Student, Cart, Order, and Review tables let students create accounts, manage carts, place orders, and rate books.
 - Customer Service module: Ticket, Ticket_category, and Ticket_state_history let CS staff create tickets, assign them to admins, and track ticket progress.
 - Administrator module: Book, Course_Book, University, Department, Course, and Instructor tables let admins manage inventory, courses, and course book requirements.
 - SuperAdmin module: Employee table with role and Account_audit table lets the SuperAdmin create or deactivate employees and track all actions.
 - Recommendations: BookKeyword, BookSubcategory, and PurchaseHistory support generating book recommendations.
 - Assumptions & Trade-offs: Any design decisions, alternatives considered, and justification.
 - Employee roles (CS, Admin, SuperAdmin) are all in the Employee table with a role field. This simplifies the design and inheritance.
 - There is only one SuperAdmin. The application enforces this.
 - Addresses are in a separate table to avoid repeating data and to keep things organized.
 - Many-to-many relationships like books-authors and courses-books are handled with junction tables to allow flexibility.
 - Dates are sometimes stored as VARCHAR for simplicity, though DATE or TIMESTAMP could enforce stricter validation.
 - Some optional or rare features are simplified (like SuperAdmin as a role rather than a separate table) to reduce complexity.

Phase 4:

1. UI STACK & RATIONALE

Technology Choice:

We decided to use MySQL as the backend database and a frontend built using Python. The UI will be a GUI built using Tkinter.

Database Connection:

The frontend will connect directly to the MySQL database using mysql-connector-python. This driver lets us execute SQL queries directly from Python functions triggered by GUI buttons.

2. SCREENS / FLOWS

Screen 1: Login Portal

- Name and Purpose:
 - “Login Portal” is the starting page for all users. Upon successful logins, it will direct them to the correct dashboard for their title (Student, Admin, or Super Admin).
- Inputs & Widgets:
 - Username/Email: Text Entry
 - Password: Masked Text Entry
 - Role Selection: Dropdown
 - Buttons: Login or exit
- Validation Rules:
 - All three fields (username, password, role) are required.
 - When the login form is submitted, the application queries either the Student or Employee table for a matching username (and technically password) and checks that the account is active.
- Success/Error Behavior:
 - Error: Show a pop-up message box: "Invalid credentials" or "User not found."
 - Success: Close Login window and open the Main Dashboard corresponding to the user's role.
- DB Interaction:
 - Read: Queries the Student table OR the Employee table based on the selected role.
 - Logic: `SELECT student_id, password FROM Student WHERE email = %s`

Screen 2: Student - Book Search & Cart

Purpose: “Student- Book Search & Cart” Allows students to search, view book details, and add items to their current cart.

- Inputs & Widgets:

- Search Bar: Text Entry for keywords/titles.
- Filter Dropdown: Filter by Category (e.g., "Computer Science", "History") populated from the Category table.
- Results Table: Table with columns Title, Author, Category, Rating, Qty, pre-populated in the sketch with a row like “Moby Dick – Herman Melville – Adventure – 5 – 4”.
- Other Buttons: Buttons under the table. View Details and Add to cart.
- Validation Rules:
 - User cannot add more copies than available in stock.
- Success/Error Behavior:
 - Success: Toast message "Added to Cart" and the cart icon counter updates.
 - Error: Message box "Out of Stock" if availability changes in real-time.
- DB Interaction:
 - Read: `SELECT * FROM Book b JOIN BookAuthor ba ON b.book_id = ba.book_id ...` to populate the list.
 - Read: `SELECT` query joining Book, Category, optional Subcategory, and Review (for average rating), with filters applied.
 - Insert: If no active cart exists, `INSERT INTO Cart`. Then `INSERT INTO Cart_Item` linking the `cart_id` and `book_id`.

Screen 3: Admin - Inventory Management (Add Book)

Purpose: “Admin - Inventory Management (Add Book)” Allows Administrators to add new book titles to the system catalog.

- Inputs & Widgets:
 - Title, Publisher, ISBN13: Text Entries.
 - Price: Numeric Entry (float).
 - Quantity: Spinbox (integer).
 - Category: Dropdown (populated from Category table).
 - Save/Update Button: Commits the change.
- Validation Rules:
 - ISBN13 must be exactly 13 characters and unique.
 - Price and Quantity must be non-negative.
 - Title and Category are required.
- Success/Error Behavior:
 - Error: Inline red text near the ISBN field if it's a duplicate.
 - Success: Clears the form and shows "Book Saved Successfully" dialog.
- DB Interaction:
 - Insert: `INSERT INTO Book (title, isbn13, price, quantity, category_id, ...)`
 - Insert (Secondary): Logic to handle BookAuthor junction table insertions if authors are selected.

Screen 4: Book Details & Reviews

Purpose: “Book Details & Reviews” show detailed book information and allow students to add the book to their cart and rate it.

- Inputs & Widgets:
 - Read-only labels for: title, authors, category, ISBN, description, price, and available quantity.
 - Numeric quantity selector (dropdown).
 - Rating input using dropdown for 1–5 stars.

- Read-only table showing existing reviews (rating + date).
 - Buttons: Add to Cart, Submit Rating, Back.
- Validation Rules:
 - Quantity must be an integer ≥ 1 and $\leq$ the current stock for that book.
 - Rating must be an integer between 1 and 5.
 - Each student may only have one rating per book; if one exists, the UI loads it and lets them update instead of creating duplicates.
- DB Interaction:
 - Reads from Book, Category, Author / BookAuthor, and Review for that book_id.
 - On Add to Cart:
 - Ensure there is an active Cart row for the student (create if needed).
 - INSERT or UPDATE into CartItem with the chosen quantity.
 - On Submit Rating:
 - INSERT or UPDATE into Review with rating and rating_date.
- Success/Error Behavior:
 - Success: Messagebox like “Added 2 copies to your cart” or “Rating saved” and the average rating display refreshes.
 - Error: Validation errors appear next to the specific field (e.g., “Quantity exceeds available stock.”).

Screen 5: Customer Support - Ticket Dashboard

Purpose: “Customer Support – Ticket Dashboard” allows students to report issues (order problems, website bugs, etc.) and monitor ticket status over time.

- Inputs & Widgets:
 - New Ticket tab
 - Ticket Category drop-down (values from TicketCategory).
 - Title text field (Entry).
 - Description text area (Text).
 - Related order drop-down
 - Buttons: Submit Ticket, Reset.
 - My Tickets tab
 - Table listing the student’s tickets: ID, category, current status, created date, last update.
 - Button View Details to open a detailed view (or show details in a panel on the side).
- Validation Rules:
 - Category, title, and description are required.
 - Description must have a minimum length (e.g., at least 10 characters) to encourage useful detail.
- DB Interaction:
 - On load:
 - SELECT ticket categories from TicketCategory.
 - SELECT the student’s previous tickets from Ticket and latest status from TicketStateHistory.
 - On submission:
 - INSERT into Ticket (student ID, category, title, description, created_at, status = “OPEN”).

- INSERT into TicketStateHistory with the starting status and null employee_id (unassigned).
- Success/Error Behavior:
 - Success: Show “Ticket #... created” message, clear the form, and refresh the “My Tickets” tab.
 - Error: Highlight missing fields or show a general “Could not create ticket” dialog if the database insert fails.

Screen 6: Employee Ticket Dashboard – Ticket List & Detail Pane

Purpose: “Employee Ticket Dashboard – Ticket List & Detail Pane” gives Admin users a centralized view of all tickets, with tools to filter, assign, and update ticket status.

- Layout:
 - Top area: filters for Status, Category, Assigned (All / Me / Unassigned) and optional date range, plus Apply / Clear buttons.
 - Left pane: Ticket Queue table with ID, student, status, category, and created_at.
 - Right pane: Ticket Details showing fields for the currently selected ticket.
- Inputs & Widgets:
 - Filter controls:
 - Status dropdown (All, Open, In Progress, Closed).
 - Category dropdown.
 - Assigned dropdown (All, Me, Unassigned).
 - Two date picker or text fields for “from” and “to” (or simple text for this prototype).
 - Buttons: Apply, Clear, Refresh.
 - Ticket queue: Treeview table.
 - Ticket detail panel:
 - Read-only labels: ticket ID, student, created_at, original description.
 - Drop-downs: Category (optional), Status (Open/In Progress/Closed), Assigned Admin (list of employees with role Admin).
 - Multiline notes text area.
 - Button Save Update.
- Validation Rules:
 - Only Admins can move tickets to Closed; Customer Support roles can leave notes and change category/status to “In Progress” but not close.
 - Status transitions must be valid (we will block impossible transitions like reopening a closed ticket in this prototype).
- DB Interaction:
 - Ticket list: SELECT joining Ticket, Student, TicketCategory, and Employee (for the assigned admin).
 - When a ticket is updated:
 - UPDATE Ticket to change status, assigned admin, and resolution fields.
 - INSERT into TicketStateHistory with old status, new status, acting employee_id, timestamp, and notes.
- Success/Error Behavior:
 - Success: Detail panel saves, the queue refreshes, and a small success message appears (e.g., “Ticket 101 updated”).
 - Error: Pop-up explaining that the update failed; no changes are committed.

3. ARTIFACTS (WIREFRAMES)

Refer to the attached screenshotted wireframes for visual representation. Other screens not depicted in the screenshots reuse this same layout structure and visual style, as described above.

Accessibility & Usability Notes

- All forms will support logical tab-key navigation (Top-Left to Bottom-Right).
- All screens use a consistent color palette

Cart + Checkout

Search

Book Fetch Welcome, Jane [0] [view Cart]

Search For Books

Search:

Category: [All v] Subcategory: [All v] ☐ only in stock ☐ Required for my courses

Search Results

Title	Author	Category	Rating	Qty
Moby-Dick	Herman Melville	Adventure	★★★★★ 5	1

Rows of Matching Books

[View Details](#) [Add to Cart](#)

Book Fetch Welcome, Jane [0] [view Cart]

Cart for Jane Doe

Title	Qty	Price	Total
Moby-Dick	1	65.00	65.00

Rows of cart items

[↑ Increase](#) [↓ Decrease](#) [Remove](#) [Proceed to checkout](#) [Cancel](#)

Order Summary

Subtotal: \$ 65

Shipping: [standard]

Total: \$ 65

New Ticket

Book Fetch Welcome, Jane [0] [view Cart]

Support Center

[New Ticket](#) [My Tickets](#)

Ticket Category: [other issue v]

Title:

Description:

Related object: [Select Recent Order v](#)

[Reset](#) [Submit Ticket](#)

existing Ticket

Book Fetch Welcome, Jane [0] [view Cart]

Support Center

[New Ticket](#) [My Tickets](#)

ID	Category	Status	Created At	Last Update
Rows of Existing Tickets				

[View Details](#)

Employee Ticket Dashboard

Book Fetch [Role: Admin] [Logout]

Employee Dashboard

Filters: Status [All v] Category [All v] Assigned [All v]

Date from [] to [] [Apply](#) [Clear](#)

ID	Student	Status
101	J. Doe	open
102	A. Smith	closed
...	...	...

[Refresh](#)

Ticket Details

Ticket ID:

Student:

Category:

Status:

Admin Assigned: [v]

Notes:

[X](#) [Save update](#)

Phase 5:

Administrator - Book Management

This screen lets an administrator maintain the core Book records that drive all inventory and reporting.

- The Create Book button inserts a new row into the BOOK table using the required fields Title, Category ID, ISBN13, Price, and Quantity, plus any optional metadata (publisher, edition, language, format, etc.).
- Update Inventory (Price & Quantity ONLY) runs an UPDATE on the existing book identified by ISBN13 and changes only price and quantity columns. This supports day-to-day stock and pricing adjustments without touching descriptive attributes.
- Update Book Details (Partial) allows an admin to modify non-inventory attributes (publisher, edition, condition, acquisition type, etc.) for an existing book while leaving price and quantity unchanged.

 Administrator Module

Book Management

Title*	
Category ID*	
Publisher	
Published Date (YYYY-MM-DD)	
Edition	
Language	
ISBN13*	
ISBN10	
Condition	
Acquisition Type	
Weight	
Price*	
Quantity*	
Format	

Administrator - University & Courses

This composite screen handles all academic-side reference data required by the system: universities, departments, and courses.

- The Create University section inserts a university into the UNIVERSITY table using University Name and Address ID as required data, and optionally stores the representative's name, email, and phone for future contact.
- The Department section lets the admin create departments tied to a specific University ID. Clicking Add Department inserts a row into DEPARTMENT with the provided name and foreign key back to the university.
- The Course section lets the admin define individual courses under a department. Add Course inserts the course into the COURSE table with a department foreign key plus course code and course name.

The screenshot displays a web interface titled "University & Courses". It is divided into three main sections:

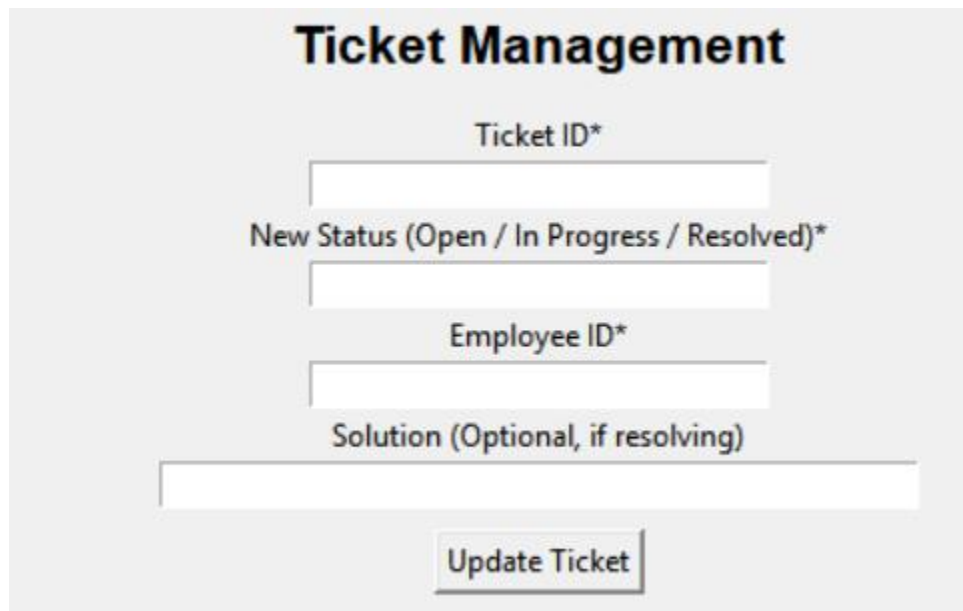
- Create University:** This section contains input fields for "University Name*", "Address ID*", "Rep First Name", "Rep Last Name", "Rep Email", and "Rep Phone". Below these fields is a button labeled "Create University".
- Department:** This section has input fields for "Department" (with a dropdown arrow), "University ID*", and "Department Name*". Below these is a button labeled "Add Department".
- Course:** This section has input fields for "Course" (with a dropdown arrow), "Department ID*", "Course Code*", and "Course Name*". Below these is a button labeled "Add Course".

Administrator - Ticket Management

The Ticket Management portion supports updates to customer-service tickets already stored in the system.

- The user supplies a Ticket ID, chooses a New Status (Open / In Progress / Resolved), and enters the Employee ID handling the ticket.
- Clicking Update Ticket executes an UPDATE on the TICKET table, changing status and assigned employee for the given ticket.

This module implements the required functionality for tracking ticket lifecycle and supports any reports that summarize open vs. resolved tickets or employee workloads.



The image shows a web form titled "Ticket Management". It contains four input fields stacked vertically: "Ticket ID*", "New Status (Open / In Progress / Resolved)*", "Employee ID*", and "Solution (Optional, if resolving)". Below these fields is a button labeled "Update Ticket".

Administrator - Course Book Association

This section creates the many-to-many relationship between courses and books.

- The admin enters a Course ID and Book ID, then selects the Usage Type (Required or Recommended).
- Create Course–Book Link inserts a row into the junction table (e.g., COURSE_BOOK) capturing this association and usage type.

These links are used by reports that list which books are required or recommended for each course and help drive ordering/stocking decisions.

Course–Book Association

Course ID*

Book ID*

Usage Type* (Required / Recommended)

☐

Create Course–Book Link

Super Administrator - Create Customer Service Employee

This screen allows a super admin to add new customer service (CS) employees to the system.

- Required attributes include first name, last name, email, and Address ID; optional attributes include phone, gender, salary, and SSN.
- Create CS Employee inserts the new employee into the EMPLOYEE (or CS_EMPLOYEE) table with the proper role and status set to active by default.

 Super Administrator Module

Super Admin Actions

Create Customer Service Employee

First Name*

Last Name*

Email*

Phone

Address ID*

Gender

Salary

SSN

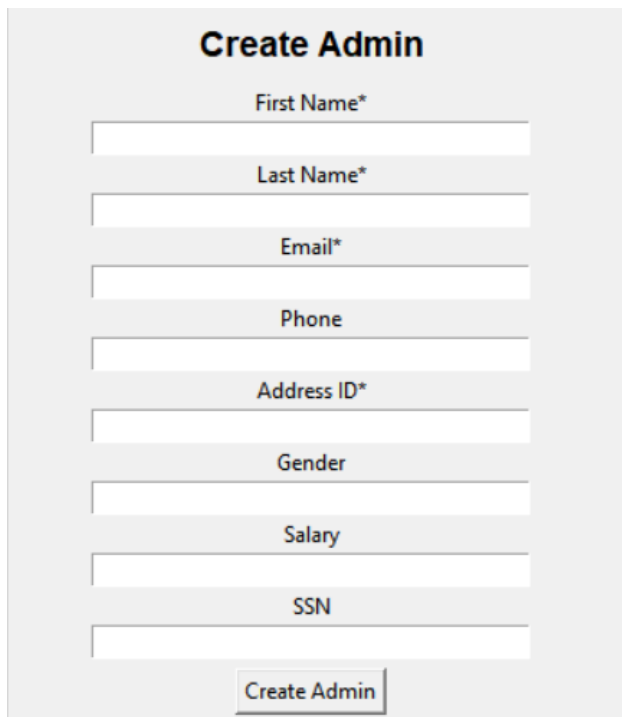
Create CS Employee

Super Administrator - Create Admin

This screen is similar to the CS employee creation form but targets administrative users.

- It captures the same personal and contact details (name, email, phone, address, gender, salary, SSN).
- Create Admin inserts a new employee row with an administrator role, enabling that user to access the administrator module screens.

This implements the functional requirement for creating new administrator accounts without building a separate login/registration subsystem.



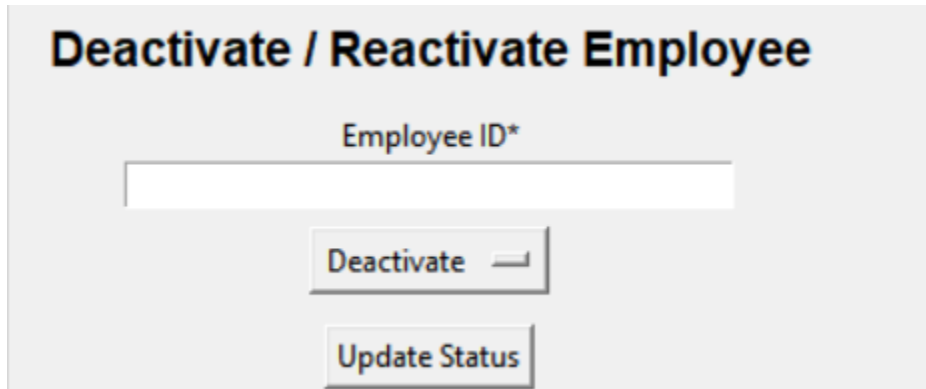
The image shows a web form titled "Create Admin". It contains ten input fields arranged vertically, each with a label above it. The labels are: "First Name*", "Last Name*", "Email*", "Phone", "Address ID*", "Gender", "Salary", "SSN", and "Create Admin" (which is a button). The fields for "First Name*", "Last Name*", "Email*", "Address ID*", "Gender", "Salary", and "SSN" are empty. The "Phone" field contains the number "1234567890". The "Create Admin" button is located at the bottom right of the form.

Super Administrator - Deactivate / Reactivate Employee

This module controls the active status of any employee account without deleting data.

- The super admin enters an Employee ID.
- Deactivate updates the employee's status flag (for example, sets `is_active = 0`), preventing that employee from being assigned new tickets or logging activity.
- Update Status reverses the flag when an employee should be reactivated (e.g., `is_active = 1`).

This design preserves referential integrity (no rows are physically deleted) while satisfying the requirement to manage employee availability within the system.wa



The screenshot shows a web form titled "Deactivate / Reactivate Employee". Below the title is a text input field labeled "Employee ID*". Underneath the input field are two buttons: "Deactivate" and "Update Status". The "Deactivate" button has a small right-pointing arrow next to it. The "Update Status" button is positioned directly below the "Deactivate" button.

Customer Service Module - Create Trouble Ticket

This screen allows a customer service employee to open a new trouble ticket on behalf of a customer or internal user. Fields marked with an asterisk (*) are required.

- Category ID* identifies the issue category and allows the ticket system to classify the request correctly.
- Title* is a short summary describing the problem.
- Description* captures the full details of the issue so the assigned admin or employee can begin troubleshooting.

Selecting Submit Ticket inserts the new ticket into the TICKET table with status set to Open by default. This satisfies the requirement for creating service tickets within the application.

Customer Service Module

Customer Service Actions

Create Trouble Ticket

Category ID*

Title*

Description*

Submit Ticket

Customer Service Module – Update Trouble Ticket

This screen allows a customer service employee to update an existing ticket after it has been created.

- Ticket ID* is required to locate the existing ticket in the system.
- Assign to Admin (Optional) allows CS staff to send the ticket directly to an administrator by updating the assigned employee/admin ID. This field is optional because not all tickets require immediate escalation.
- Description Update allows the CS employee to append or replace ticket notes, add additional details, or document progress.

Selecting Update Ticket performs an UPDATE operation on the TICKET table, modifying only the provided fields. No required fields other than Ticket ID* must be supplied for this operation.

UI pages

Update Trouble Ticket

Ticket ID*

Assign to Admin (Optional)

Description Update

Allows Admins to choose a ticket and assign it to an employee and update the description of the ticket.

Add Book Review / Rating

Create a rating for a book (1-5).
Rating_date is set automatically.

Student ID

Book ID

Rating (1-5)

Allows employees to create new student accounts as well as carts for those students

Add Book to Cart

Cart ID

Book ID

Quantity

Place Order from Cart

Cart ID

Shipping Type

Card Name

Card Number

Card Type

Card Exp (YYYY-MM-DD)

Allows students to add books to their carts and place their orders when finished

Update Cart Item (Change Quantity / Remove)

Set new quantity for an item in a cart.
If new quantity is 0, the item will be removed
and inventory will be restored.

Cart ID

Book ID

New Quantity

Cancel Order

Cancel an order and return items to inventory.
Allowed only if order is not Shipped/Completed/Cancelled.

Order ID

Student ID (optional)

Allows students to modify their carts as well as cancel orders if they made a mistake.